

将者，智、信、仁、勇、严也。

大智若愚，大巧若拙，大勇若怯！

公告

昵称：bonelee
园龄：2年11个月
粉丝：226
关注：0
+加关注

<2019年7月>

日	一	二	三	四	五	六
30	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31	1	2	3
4	5	6	7	8	9	10

搜索

找找看

谷歌搜索

常用链接

我的随笔

我的评论

我的参与

最新评论

我的标签

我的标签

安全分析(276)

机器学习(208)

leetcode(163)

elasticsearch(115)

python(110)

数据库(108)

算法(90)

搜索引擎(85)

其他(80)

深度学习(70)

更多

随笔档案

2019年7月 (18)

2019年6月 (13)

2019年5月 (16)

2019年4月 (17)

2019年3月 (7)

2019年2月 (9)

2019年1月 (14)

2018年12月 (13)

2018年11月 (35)

2018年10月 (28)

2018年9月 (29)

2018年8月 (17)

2018年7月 (41)

2018年6月 (62)

2018年5月 (85)

2018年4月 (66)

2018年3月 (113)

2018年2月 (33)

2018年1月 (46)

facebook开源的prophet时间序列预测工具---识别多种周期性、趋势性（线性，logistic）、节假日效应，以及部分异常值

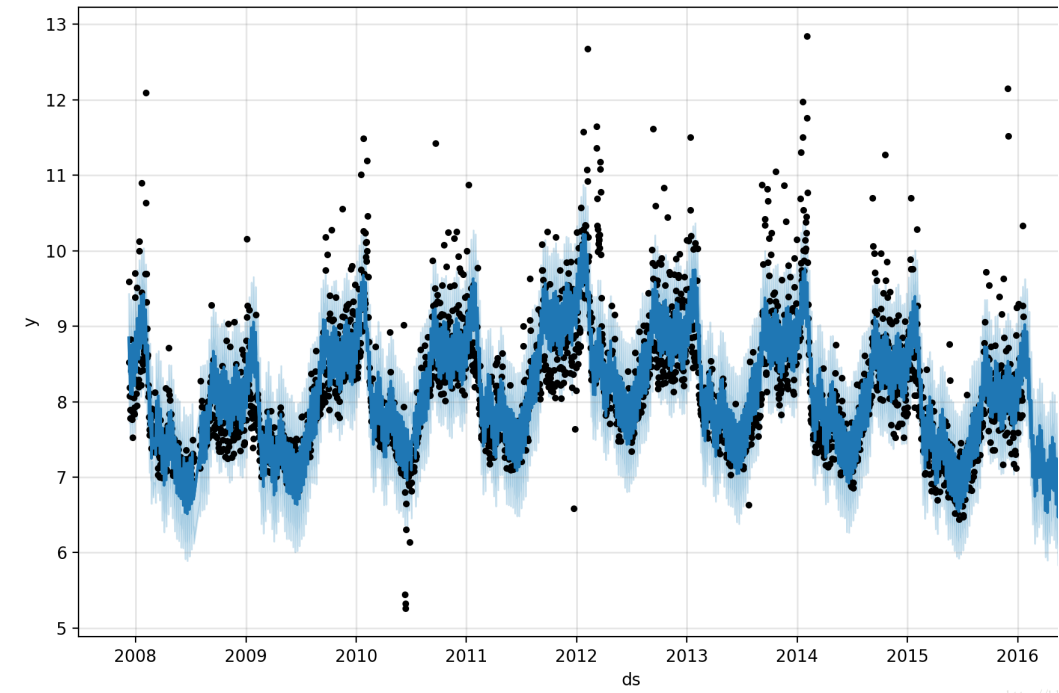
简单使用

代码如下

这是官网的quickstart的内容，csv文件也可以下到，这个入门以后后面调试加入其它参数就很简单了。

```
1 import pandas as pd
2 import numpy as np
3 from fbprophet import Prophet
4 import matplotlib.pyplot as plt
5
6 df = pd.read_csv('prophet2.csv')
7 df['y'] = np.log(df['y'])
8 df.head()
9 m = Prophet()
10 m.fit(df);
11 future = m.make_future_dataframe(periods=365)
12 future.tail()
13 forecast = m.predict(future)
14 print(forecast[['ds', 'yhat', 'yhat_lower', 'yhat_upper']])
15 m.plot(forecast)
16
17 x1 = forecast['ds']
18 y1 = forecast['yhat']
19
20
21 plt.plot(x1,y1)
22 plt.show()
```

运行结果如下图



- 2017年12月 (37)
- 2017年11月 (81)
- 2017年10月 (25)
- 2017年9月 (37)
- 2017年8月 (43)
- 2017年7月 (53)
- 2017年6月 (40)
- 2017年5月 (76)
- 2017年4月 (50)
- 2017年3月 (94)
- 2017年2月 (113)
- 2017年1月 (107)
- 2016年12月 (80)
- 2016年11月 (38)
- 2016年10月 (11)
- 2016年9月 (3)
- 2016年8月 (3)

最新评论

1. Re:如何利用Ai识别未知——加入未知类（不太靠谱），检测待识别数据和已知样本数据的匹配程度（例如使用CNN降维，再用knn类似距离来实现），将问题转化为特征搜索问题而非决策问题，使用HTM算法（记忆+模式匹配预测就是智能），GAN异常检测未知恶意代码的深度学习检测方法Abstract本发明公开了信息安全技术领域的未知恶意代码的深度学习检测方法。它包括下列步骤：1)利用字节级n元文法提取训练集中文件的特征向量；2)构建HTM网络结构，并.....

--bonelee

2. Re:如何利用Ai识别未知——加入未知类（不太靠谱），检测待识别数据和已知样本数据的匹配程度（例如使用CNN降维，再用knn类似距离来实现），将问题转化为特征搜索问题而非决策问题，使用HTM算法（记忆+模式匹配预测就是智能），GAN异常检测

--bonelee

3. Re:LSTM如何解决梯度消失或爆炸的？使得单元间的传递 $\frac{\partial S_j}{\partial S_{j-1}}$ 为0或1，这个不对吧？

--WOTGL

4. Re:spark context stop use with as with 的一般执行过程一段基本的with表达式，其结构是这样的：with EXPR as VAR: BLOCK其中：EXPR可以是任意表达式；as VAR是可选的。其一般的执行过程是这样的：计.....

--bonelee

5. Re:时间序列预测——深度好文，ARIMA是最难用的（数据预处理过程不适合工业应用），线性回归模型简单适用，预测趋势很不错，xgboost的话，不太适合趋势预测，如果数据平稳也可以使用。博主，几点疑问请教下：1.关于差分，个人理解是消除趋势因素，而移动平均法则是消除周期因素？不知道是否正确？还有，关于差分，d表示差分的阶数。对于d = 0, yt =Yt, d =1 yt = Yt -Yt-1.....

--老笨啊

from : <https://www.zhihu.com/question/56585493/answer/234300942>

在今年三月prophet刚发布的时候就简单用过，但最近才想起去读paper.....

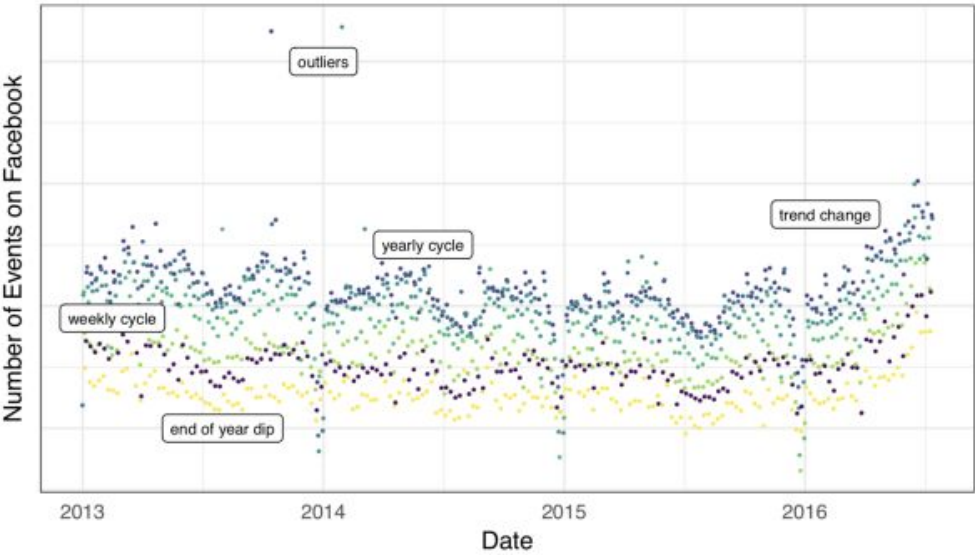
首先，prophet是一个工业级应用，而不是说在时间序列预测的模型上有非常大的创新。

记得今年在参加一次猫眼电影票房预测的内部分享时，旁边坐了一个外卖的PM。结束时对方问我，有什么方法可以预测外卖的订单量。我当时想了想，诸如Holt-Winters指数平滑、ARIMA、或者deep learning的LSTM，似乎都不是那么容易解释。

时间序列预测对大部分公司而言都存在必要的需求，比如电商预测GMV，外卖O2O预测成交量波动以便于运力分配，酒店预测间夜量来调整定价与销售，等等。但通常而言，时间序列预测对不少公司来说是一个难题。主要原因除了时间序列预测本身就是玄学（大雾）之外，还要求分析师同时具备深厚的专业领域知识（domain knowledge）和时间序列建模的统计学知识。此外，时间序列模型的调参也是一个比较复杂而繁琐的工作。

prophet就是在这样的背景下的产物，将一些时间序列建模常见的流程与参数default化，来使不太懂统计的业务分析师也能够针对需求快速建立一个相对可用的模型。

很多商业行为是存在一定的时间相依的模式。作者以Facebook上用户创造“事件”（events）来举例：



可以看到用户创造事件的数量有很明显的时间序列特征：多种周期性、趋势性、节假日效应，以及部分异常值。

然后作者用R的forecast包里的几种常见的时间序列预测技术（ARIMA, 指数平滑等等）来建模，效果惨不忍睹：

阅读排行榜

- 1. sklearn的train_test_split，果然很好用啊！（70177）
- 2. 安装和使用访问暗网(60335)
- 3. GAN的原理入门(58255)
- 4. TSNE——目前最好的降维方法(41739)
- 5. numpy中的ndarray方法和属性(37706)

评论排行榜

- 1. CC 攻击检测研究现状(45)
- 2. 网络流量预测 国内外研究现状【见评论】——传统的ARIMA、HMM模型，目前LSTM、GRU、CNN应用较多，貌似小波平滑预处理步骤非常关键(42)
- 3. 信令风暴研究现状总结(40)
- 4. ES业界优秀实践案例汇总(28)
- 5. 国外DDoS产品的一些调研——Akamai Arbor Networks Cloudflare DOSarrest F5 Fastly Imperva Link11 Neustar Nexusguard Oracle (Dyn) Radware Verisign(22)

推荐排行榜

- 1. GAN的原理入门(6)
- 2. MySQL索引实现(4)
- 3. 机器学习 LR中的参数迭代公式推导——极大似然和梯度下降(3)
- 4. 安装和使用访问暗网(3)
- 5. lucene底层数据结构——FST，针对field使用列存储，delta encode压缩doc ids数组，LZ4压缩算法(3)

Copyright ©2019 bonelee

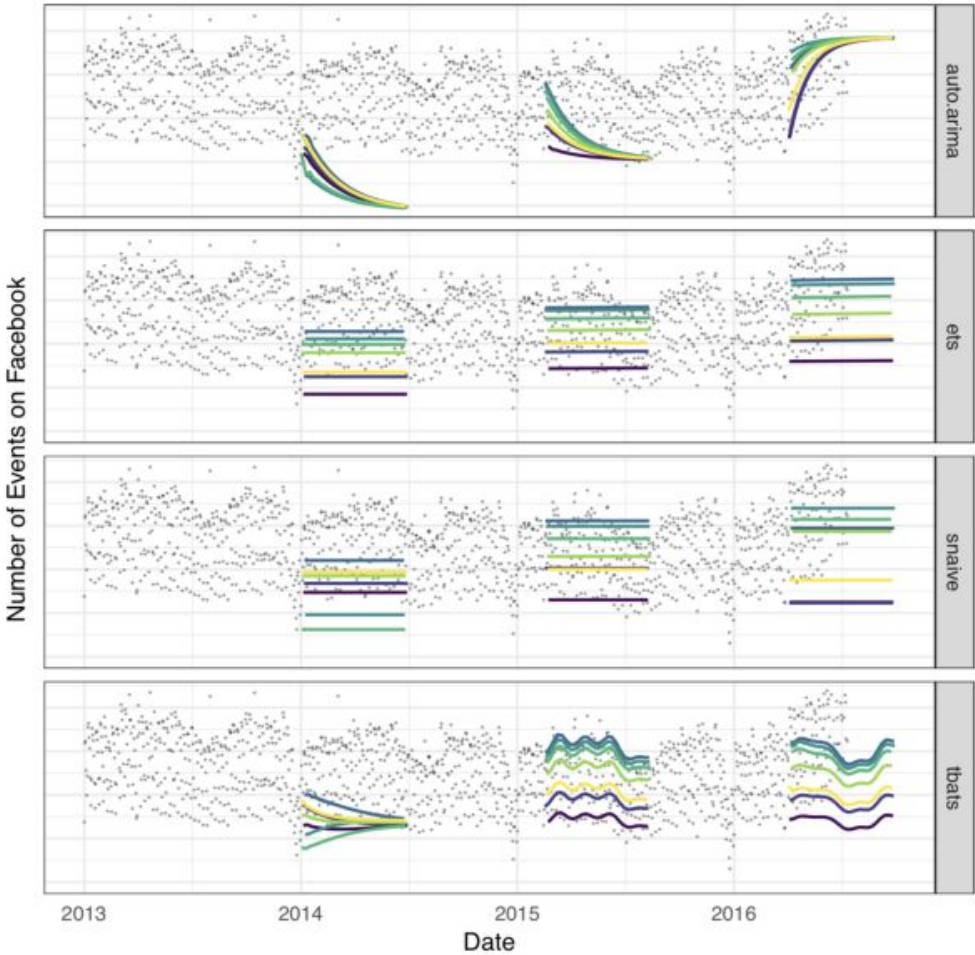


图1是ARIMA，图2是指数平滑，图3是snaive，图4是tbats。

模型结构

Prophet的本质是一个可加模型，基本形式如下：

$$y(t) = g(t) + s(t) + h(t) + \epsilon_t$$

其中 $g(t)$ 是趋势项， $s(t)$ 是周期项， $h(t)$ 是节假日项， ϵ_t 是误差项并且服从正态分布。

趋势模型

prophet里使用了两种趋势模型：饱和增长模型（ saturating growth model ）和分段线性模型（ piecewise linear model ）。两种模型都包含了不同程度的假设和一些调节光滑度的参数，并通过选择变化点（ changepoints ）来预测趋势变化。具体推导就不写了，只写下最终形式：

saturating growth model:

$$g(t) = \frac{C(t)}{1 + \exp(-(k + a(t)^T \delta)(t - (m + a(t)^T \gamma)))}$$

piecewise linear model :

$$g(t) = (k + a(t)^T \delta)t + (m + a(t)^T \gamma)$$

周期模型

prophet用傅里叶级数（ Fourier series ）来建立周期模型：

$$s(t) = \sum_{n=1}^N (a_n \cos(\frac{2\pi nt}{P}) + b_n \sin(\frac{2\pi nt}{P}))$$

对N的调节起到了低通滤波（ low-pass filter ）的作用。作者说对于年周期与星期周期，N分别选取为10和3的效果比较好。

节假日与突发事件模型

节假日需要用户事先指定，每一个节假日都包含其前后的若干天。模型形式如下（感觉就是一个虚拟变量）：

$$Z(t) = [1(t \in D_1), \dots, 1(t \in D_L)]$$

$$h(t) = Z(t)\kappa$$

$$\kappa \sim Normal(0, \nu^2)$$

模型性能

还是使用上面Facebook的例子，作者给出了Prophet的模型拟合与预测能力：

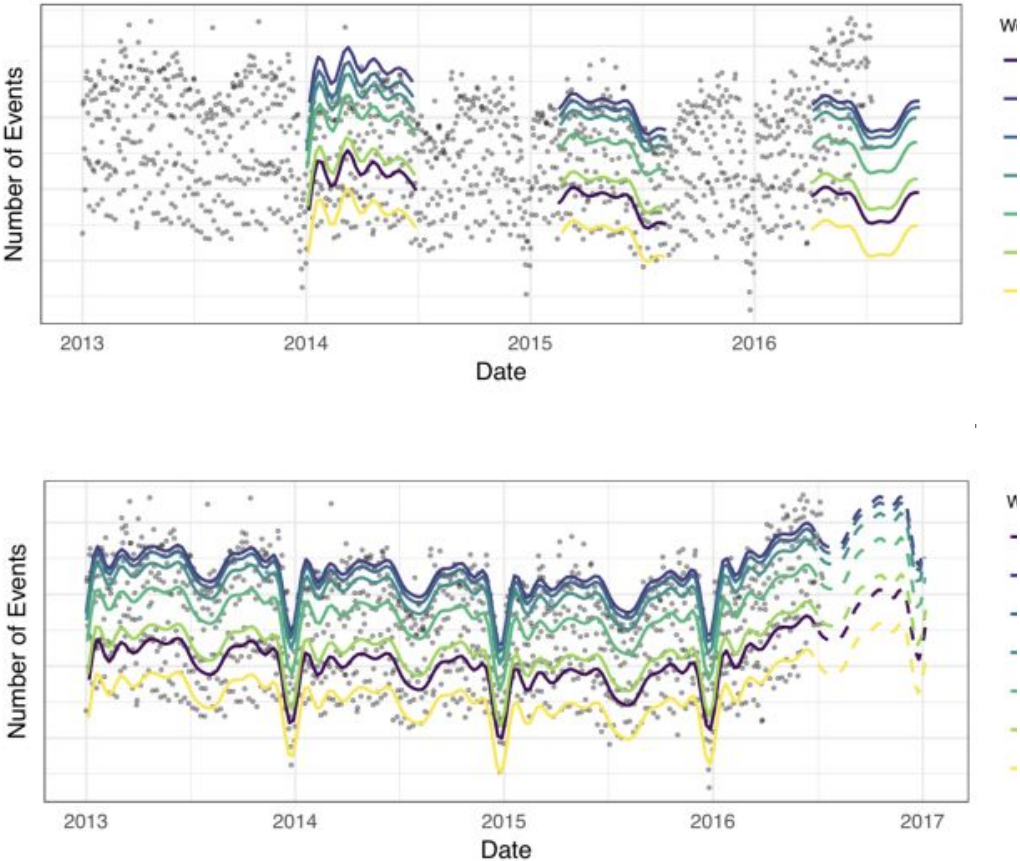
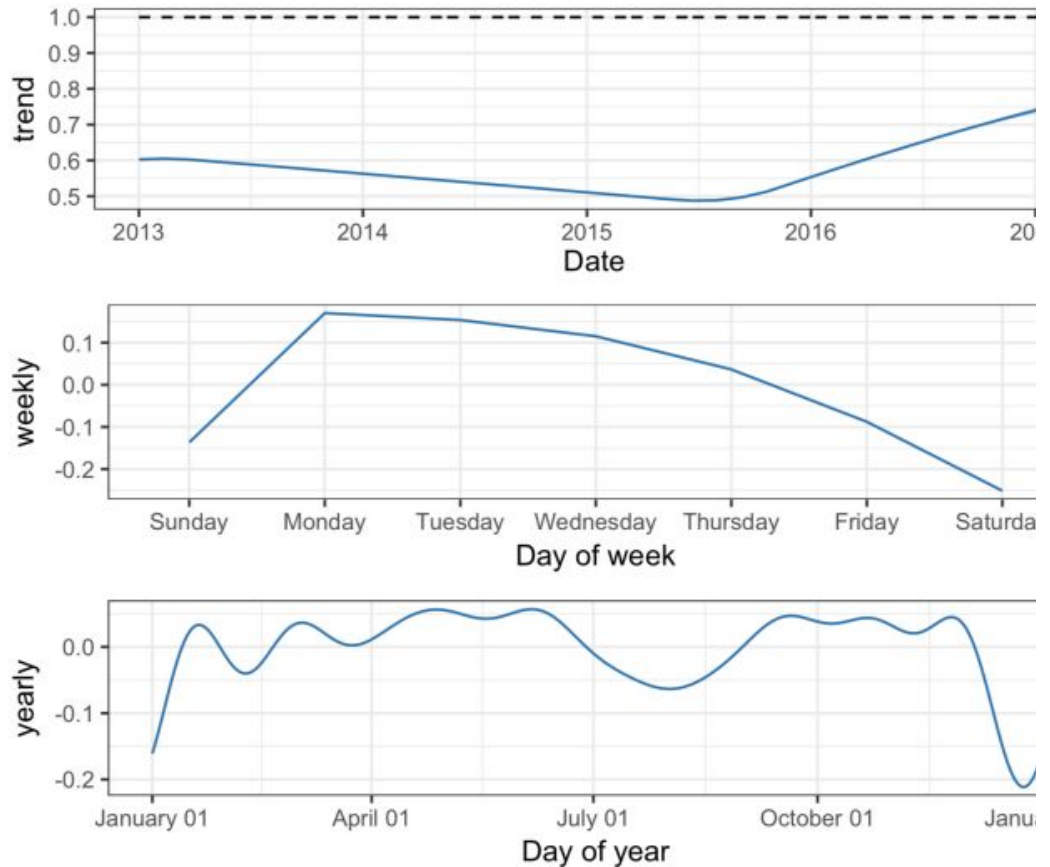


Figure 5: Prophet forecast using all available data, including the interpolation of torical data. Solid lines are in-sample fit, dashed lines are out-of-sample forecast.

看起来比前面用R的forecast做的效果好了很多，并且不需要使用者具有很强的统计背景就能够轻松进行建模。

同时prophet支持将模型分解为单独的各项组成部分，并且实现起来很容易，只需要调用一行代码 prophet_plot_components：



适用范围

很明显，Prophet只适用于具有**明显的内在规律（或者说，模式）**的商业行为数据。

虽然官方案例里通常使用日数据的序列，但对于更短时间频段，比如小时数据，也是支持的。

但对于不具有明显趋势性、周期性的时间序列，使用Prophet进行预测就不适合了。比如前面有同学用Prophet来预测沪深300.....先不说有效市场假说（EMH）否定了历史数据对未来价格拟合的可能性，就算市场存在模式，也不是能够被一个通用模型简单的线性分解成趋势和周期的。

我自己最早是基于内部历史数据，尝试公司风控的潜在损失做一个简单预测，但很明显，没有任何证据能说明过去的序列特征（比如风险集中趋势，外部环境影响，公司层面的合并等等）会在2017年重演。所以充其量就是拿来写周报，以及为2017年风控预算做一点微小的贡献.....

总结

Prophet是一个比较好用的预测工具，特别是对我这种拿着forecast的ets和auto.arima也懒得自动定阶和模型选择的人来说（逃.....

对业务分析师很友好，因为原理很简单，有R和python的基础上上手也很容易。

通常能够给出一个还不错的预测结果。比如我就对某些业务线的交易数据跑了下预测，发现大部分都能work，诸如“春节效应”这种中国特色也能抓得比较准。

from : https://blog.csdn.net/sinat_26917383/article/details/57419862

比较赞的功能点：

- 1、**大规模、细粒度数据**。其实并不是大量数据，而是时间粒度可以很小，在学校玩的计量大多都是“年/月”粒度，而这个包可以适应“日/时”级别的，具体的见后面的案例就知道了。不过，预测速度嘛~可以定义为：较慢！！
- 2、**趋势预测+趋势分解**，最亮眼模块哟~~
拟合的有两种趋势：线性趋势、logistic趋势；趋势分解有很多种：Trend趋势、星期、年度、季节、节假日，同时也可以看到节中、节后效应。
- 3、**突变点识别+调整**。多种对抗突变办法以及调节方式。
- 4、**异常值/离群值检测**。时间维度的异常值检测。突变点和异常点既相似、又不同。
- 5、**处理缺失值数据**。这里指的是你可能有一些时间片段数据的缺失，之前的做法是先插值，然后进行预测（一些模型不允许断点），这里可以兼顾缺失值，同时也达到预测的目的。可以处理缺失值数据，这点很棒。

prophet应该就是我一直在找的，目前看到最好的营销活动分析的预测工具，是网站分析、广告活动分析的福音，如果您看到本篇文章内的方法，您在使用中发现什么心得，还请您尽量分享出来~

```
# install.packages('prophet')
library(prophet)
library(dplyr)
```

一、趋势预测+趋势分解

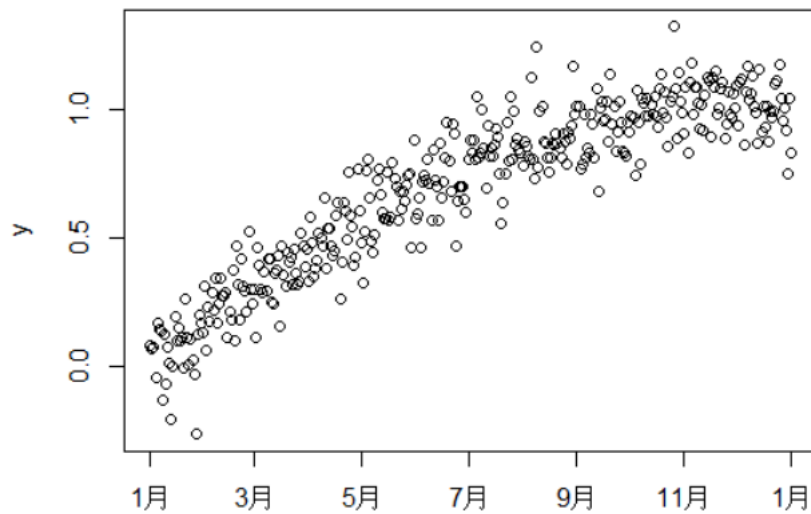
1、案例一：线性趋势+趋势分解

- 数据生成+建模阶段

```
history <- data.frame(ds = seq(as.Date('2015-01-01'), as.Date('2016-01-01'), by = 'd'),
                      y = sin(1:366/200) + rnorm(366)/10)

m <- prophet(history, growth = "linear")
```

其中，生成数据的时候注意，最好用ds（时间项）、y（一定要numeric）这两个命名你的变量，本案例是单序列+时间项。数据长这样：



http://dslog.csdn.net/sinat_26917383

prophet是生成模型阶段，m中有很多参数，有待后来人慢慢研究。

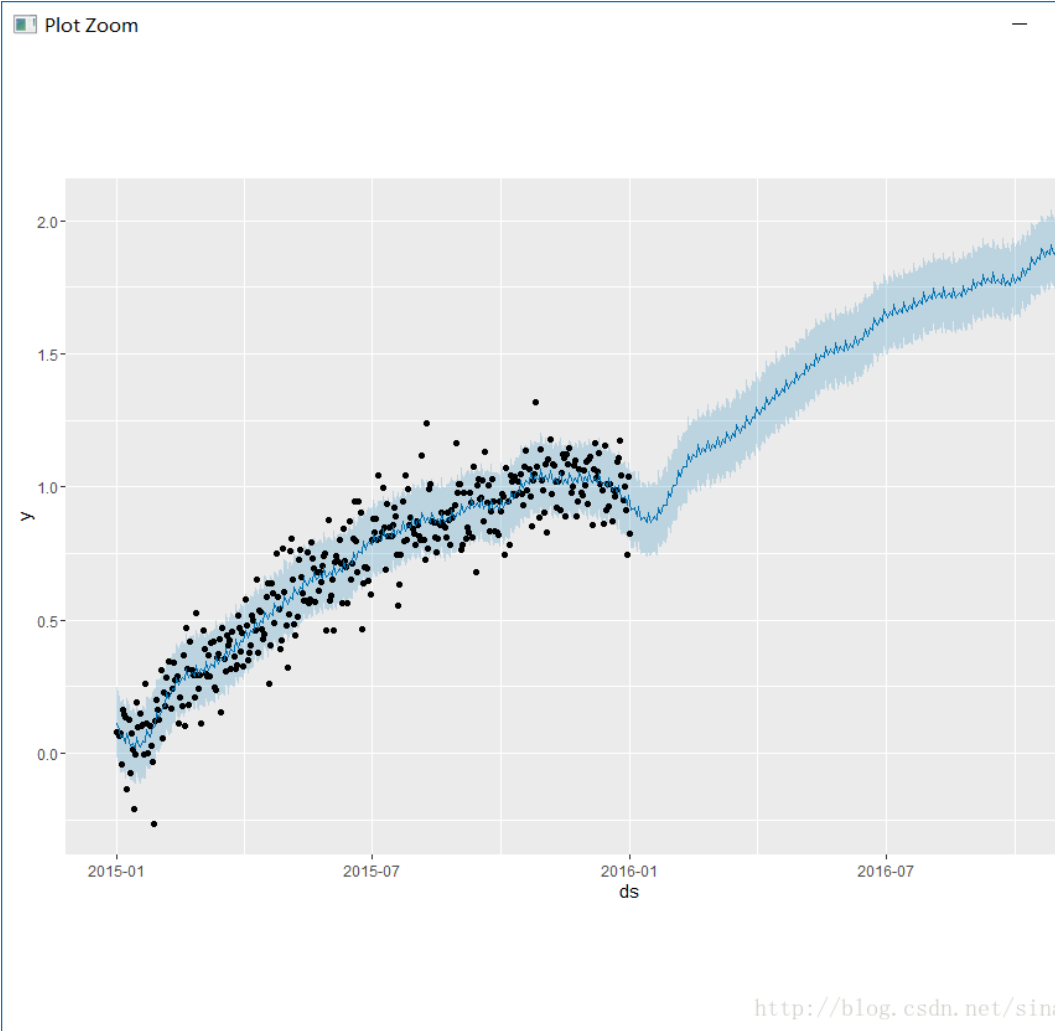
- 预测阶段

```
#时间函数
future <- make_future_dataframe(m, periods = 365)
tail(future)
#预测
forecast <- predict(m, future)
tail(forecast[c('ds', 'yhat', 'yhat_lower', 'yhat_upper')])
#直线预测
plot(m, forecast)
#趋势分解
prophet_plot_components(m, forecast)
```

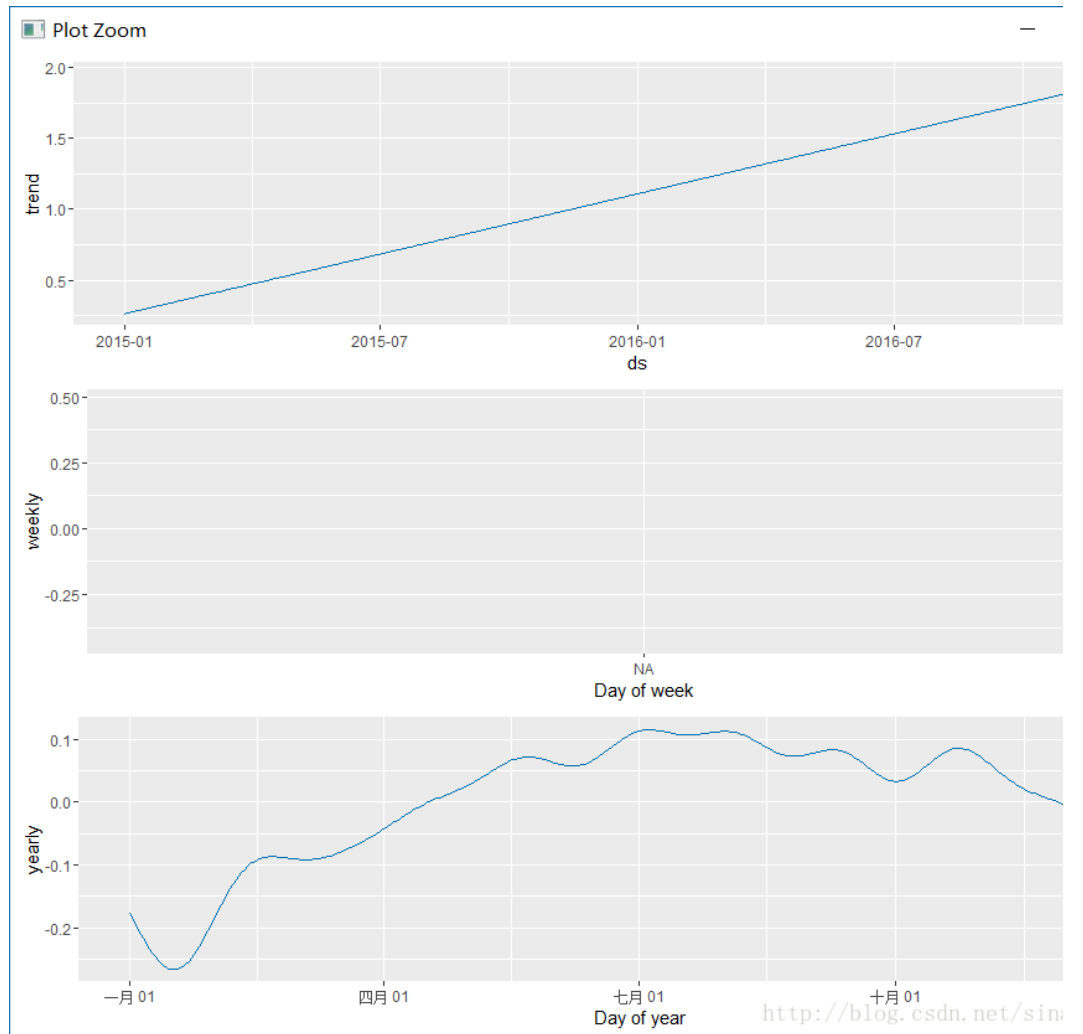
make_future_dataframe：有趣的时间生成函数，之前的ds数据是2015-1-1到2016-1-1，现在生成了一个2015-1-1到2016-12-30序列，多增加了一年，以备预测。而且可以灵活的调控是预测天，还是周，freq参数。

predict，预测那么ds是时间，yhat是预测值，lower和upper是置信区间。

感受一下plot：



prophet_plot_components函数是趋势分解函数，将趋势分成了趋势项、星期、年份，这是默认配置。



2、案例二：logitics趋势+趋势分解

logitics是啥？不懂烦请百度。

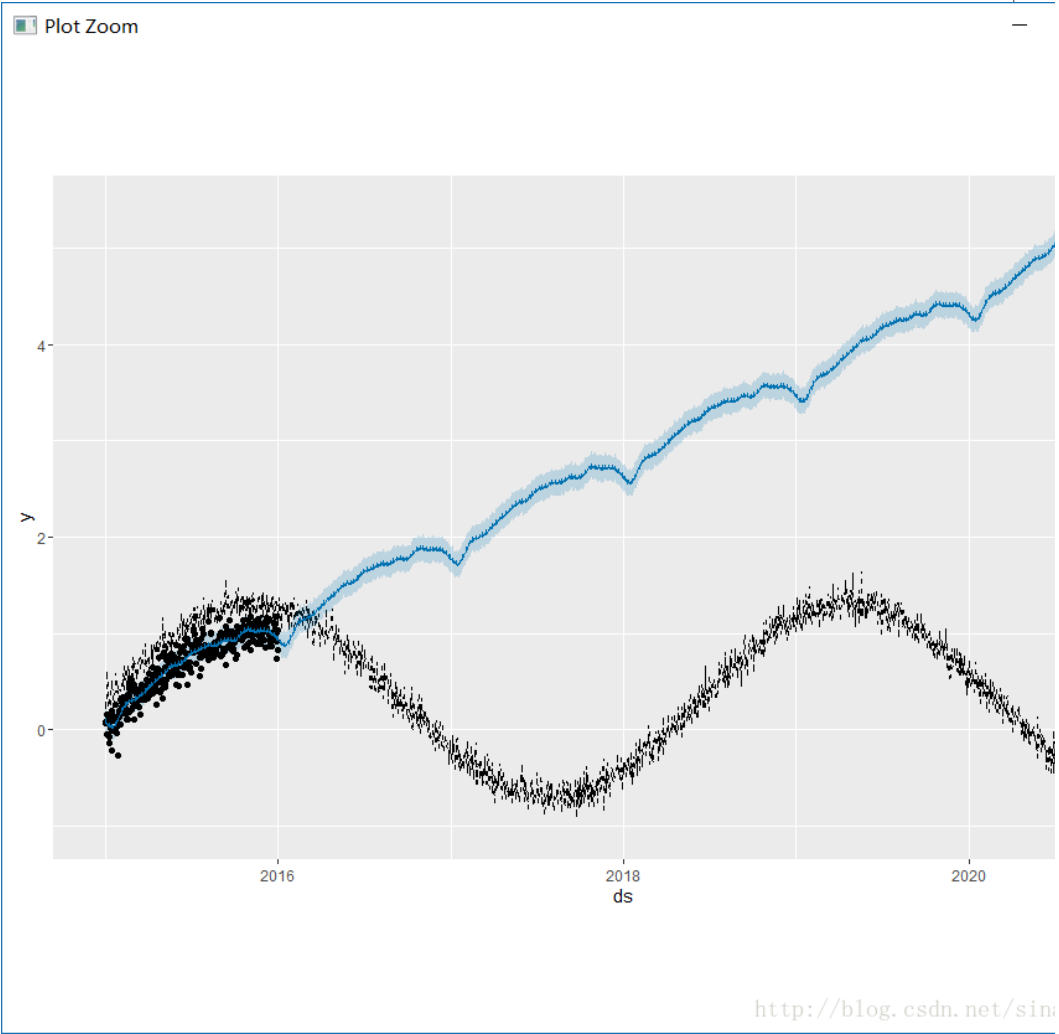
```
#数据生成阶段
history <- data.frame(ds = seq(as.Date('2015-01-01'), as.Date('2016-01-01'), by = 'd'),
                      y = sin(1:366/200) + rnorm(366)/10,
                      cap=sin(1:366/200) + rnorm(366)/10+rep(0.3,366))
#最大增长趋势，cap设置cap，就是这个规模的顶点，y当时顶点

#模型生成
m <- prophet(history,growth = "logistic")
future <- make_future_dataframe(m, periods = 1826)
future$cap <- sin(1:2192/200) + rnorm(2192)/10+rep(0.3,2192)

#预测阶段
fcst <- predict(m, future)
plot(m, fcst)
```

prophet这里如果是要拟合logitics趋势，就需要一个cap变量，这个变量是y变量的上限（譬如最大市场规模），因为y如果服从logitics趋势不给范围的话，很容易一下预测就到顶点了，所以cap来让预测变得不那么“脆弱”...

下面来看一个失败拟合logitics案例：



二、节假日效应

可以考察节中、节后效应。来看看paper中如何解释节日效应的（[论文地址](#)）：

Incorporating this list of holidays into the model is made straightforward by assuming that the effects of holidays are independent. For each holiday i , let D_i be the set of past and future dates for that holiday. We add an indicator function representing whether time t is during holiday i , and assign each holiday a parameter κ_i which is the corresponding change in the forecast:

$$h(t) = \sum_{i=1}^L \kappa_i \mathbf{1}(t \in D_i). \tag{7}$$

We can generalize Equation 7 to include effects for a window of days around a particular holiday. For instance, people behave differently in the days leading up to Christmas and after it is over. To account for that we include additional parameters for the days surrounding the holiday, essentially treating each of the days in the window around the holiday as a holiday itself.

Holidays are implemented in a similar way as seasonality, by generating a matrix of regressors

$$Z(t) = [\mathbf{1}(t \in D_1), \dots, \mathbf{1}(t \in D_L)]$$

and taking

$$h(t) = Z(t)\kappa. \tag{8}$$

As with seasonality, we use a prior $\kappa \sim \text{Normal}(0, \nu)$.

http://blog.csdn.net/sinat_26917383

也就是说，节日效应能量函数 $h(t)$ 由两部分组成， $Z(t)$ 是一个示性函数的集合（indicator function），而参数 κ 服从 $(0, \nu)$ 正态分布。可以说，将节日看成是一个正态分布，把活动期间当做波峰，`lower_window` 以及 `upper_window` 的窗口作为扩散。

1、节中效应

```
#数据生成：常规数据
history <- data.frame(ds = seq(as.Date('2015-01-01'), as.Date('2016-01-01'), by = 'd'),
                      y = sin(1:366/200) + rnorm(366)/10,
                      cap=sin(1:366/200) + rnorm(366)/10+rep(0.3,366))

#数据生成：节假日数据
library(dplyr)
playoffs <- data_frame(
  holiday = 'playoff',
  ds = as.Date(c('2008-01-13', '2009-01-03', '2010-01-16',
                 '2010-01-24', '2010-02-07', '2011-01-08',
                 '2013-01-12', '2014-01-12', '2014-01-19',
                 '2014-02-02', '2015-01-11', '2016-01-17',
                 '2016-01-24', '2016-02-07')),
  lower_window = 0,
  upper_window = 1
)
superbowls <- data_frame(
  holiday = 'superbowl',
  ds = as.Date(c('2010-02-07', '2014-02-02', '2016-02-07')),
  lower_window = 0,
  upper_window = 1
)
holidays <- bind_rows(playoffs, superbowl)

#预测
m <- prophet(history, holidays = holidays)
forecast <- predict(m, future)

#影响效应
forecast %>%
  select(ds, playoff, superbowl) %>%
  filter(abs(playoff + superbowl) > 0) %>%
  tail(10)

#趋势组件
prophet_plot_components(m, forecast);
```

数据生成环节有两个数据集要生成，一批数据是常规的数据（譬如流量），还有一个是节假日的时间数据。其中lower_window，upper_window 可以理解为假日延长时限，国庆和元旦肯定休息时间不一致，设置地很人性化，譬如**圣诞节的平安夜+圣诞节两天**，那么就要设置（lower_window = -1, upper_window = 1）。这个lower_window 的尺度为天，所以如果你的数据是星期/季度，需要设置-7/+7，比较合理。举一个python中的设置方式（时序是by week）：

```
c3_4 = pd.DataFrame({
    'holiday': 'c1',
    'ds': pd.to_datetime(['2017/2/26',
                          '2017/3/5']),
    'lower_window': -7,
    'upper_window': 7,
})
```

lower_window，upper_window 是节日效应的精髓，一般情况下，在-7 / +7 的时间跟活动期的数值不一样，刚好可以很多表示出节日的正态效应。

数据长这样：

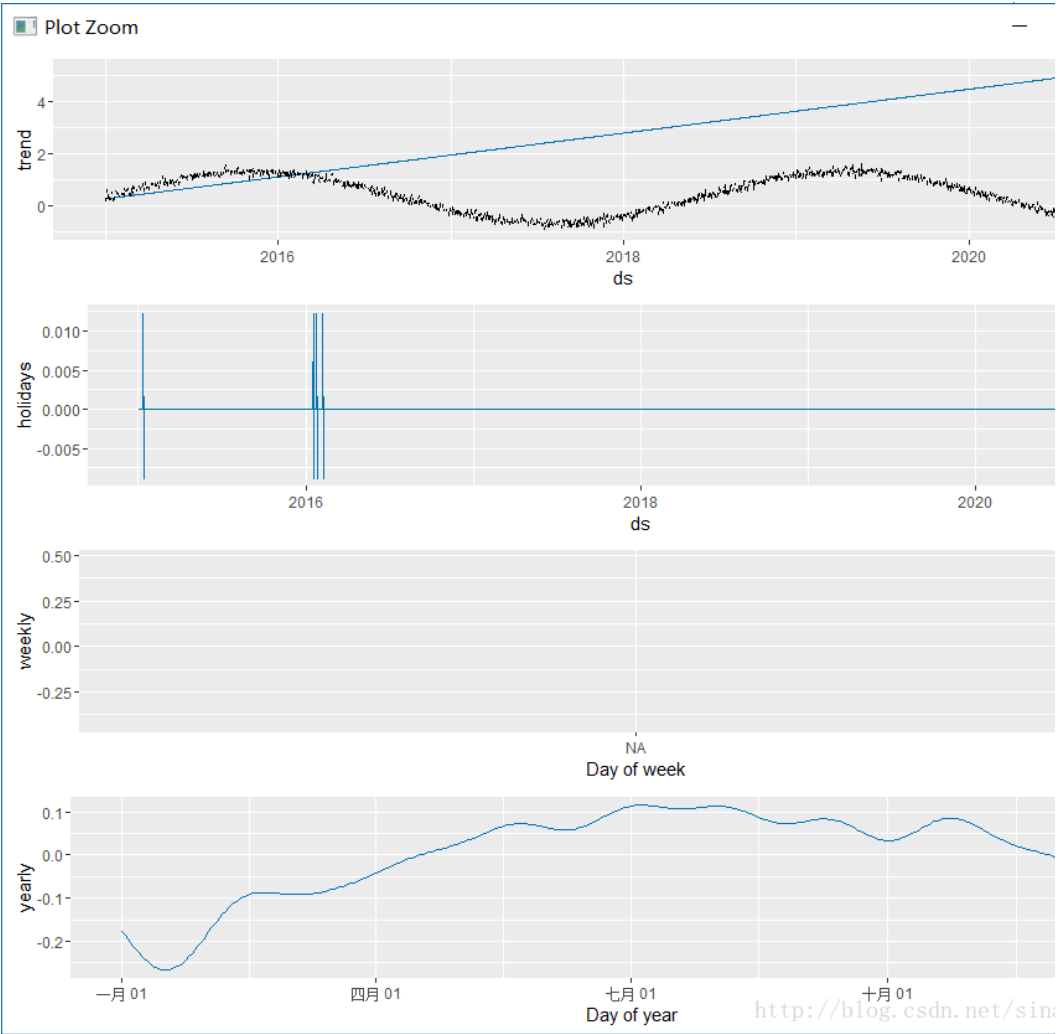
	holiday	ds	lower_window	upper_window
	<chr>	<date>	<dbl>	<dbl>
1	playoff	2008-01-13	0	1
2	playoff	2009-01-03	0	1
3	playoff	2010-01-16	0	1
4	playoff	2010-01-24	0	1
5	playoff	2010-02-07	0	1

预测阶段，记得要开启prophet(history, holidays = holidays)中的holidays。现在可以来看看节假日效应：

	ds	playoff	superbowl
1	2015-01-11	0.012300004	0
2	2015-01-12	-0.008805914	0
3	2016-01-17	0.012300004	0
4	2016-01-18	-0.008805914	0
5	2016-01-24	0.012300004	0
6	2016-01-25	-0.008805914	0
7	2016-02-07	0.012300004	0
8	2016-02-08	-0.008805914	0

从数据来看，可以看到有一个日期是重叠的，超级碗+季后赛在同一天，那么这样就会出现**节日效应累加的情况**。可以看到季后赛当日的影响比较明显，超级碗当日基本没啥影响，当然了，这些数据都是我瞎编的，要是有效应就见xxx。

趋势分解这里，除了趋势项、星期、年份，多了一个节假日影响，看到了吗？



2、调和节日效应（Prior scale for holidays and seasonality）

一些情况下节假日会发生过拟合，那么可以使用holidays.prior.scale参数来进行调节，使其平滑过渡。（不知道翻译地对不对，本来刚开始以为是节后效应...）

```
#节后效应 holidays.prior.scale
m <- prophet(history, holidays = holidays, holidays.prior.scale = 1)
forecast <- predict(m, future)
forecast %>%
  select(ds, playoff, superbowl) %>%
  filter(abs(playoff + superbowl) > 0) %>%
  tail(10)
```

主要通过holidays.prior.scale来实现，默认是10。由于笔者乱整数据，这里显示出效应，所以粘贴官网数据。官网的案例里面，通过调节，使得当晚超级碗的效应减弱，兼顾了节前的情况对当日的影响。

同时除了节前，还有季节前的效应，通过参数seasonality_prior_scale 调整

DS	PLAYOFF	SUPERBOWL	
2190	2014-02-02	1.362312	0.693425
2191	2014-02-03	2.033471	0.542254
2532	2015-01-11	1.362312	0.000000
2533	2015-01-12	2.033471	0.000000
2901	2016-01-17	1.362312	0.000000
2902	2016-01-18	2.033471	0.000000
2908	2016-01-24	1.362312	0.000000

三、突变点调节、间断点、异常点

本节之后主要就是玩案例里面的数据，案例数据如果R包中没有，可以从[这里](#)下载。

1、Prophet——自动突变点识别

时间序列里面的很可能存在突变点，譬如一些节假日的冲击。Prophet会自动检测这些突变点，并进行适当的调整，但是机器判断会出现：没有对突变点进行调整、突变点过度调整两种情况，如果真的突变点出现，也可以通过函数中的参数进行调整。

Prophet自己会检测一些突变点，以下的图就是Prophet自己检测出来的，虚纵向代表突变点。检测到了25个，那么Prophet的做法跟L1正则一样，“假装”删掉看不见这些突变。

Prophet通过首先指定大量的变化点来检测转换点，在这些变化点上允许改变速率。然后对速率变化的幅度进行稀疏优先（相当于L1正则化）-这实质上意味着先知有很多可能改变速率的地方，但是会尽可能少地使用它们。从快速入门考虑Peyton Manning预测。默认情况下，先知指定了25个潜在的变化点，它们被均匀地放置在时间序列的前80%个。

By default changepoints are only inferred for the first 80% of the time series in order to have plenty of runway for projecting the trend forward and to avoid overfitting fluctuations at the end of the time series. This default works in many situations but not all, and can be change using the changepoint_range argument. For example, `m = Prophet(changepoint_range=0.9)` in Python or `m <- prophet(changepoint.range = 0.9)` in R will place potential changepoints in the first 90% of the time series.

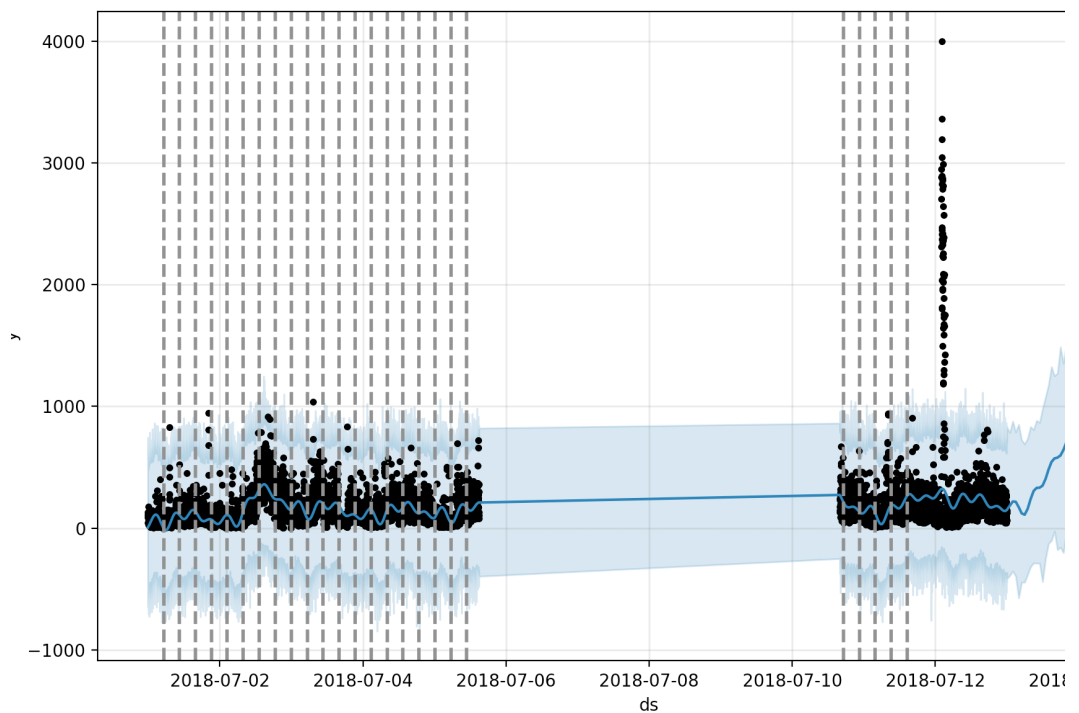
可以参考 `trend_changepoints.ipynb` 绘图看。貌似是通过变化率来检测change point。

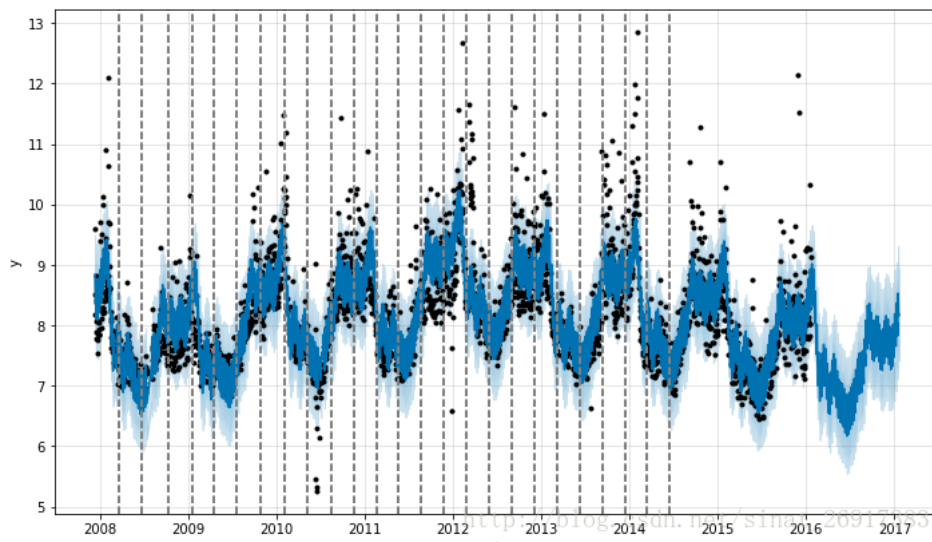
If the trend changes are being overfit (too much flexibility) or underfit (not enough flexibility), you can adjust the strength of the sparse prior using the input argument `changepoint_prior_scale`. By default, this parameter is set to 0.05. Increasing it will make the trend more flexible

从这段话可以看到，change point的参数：

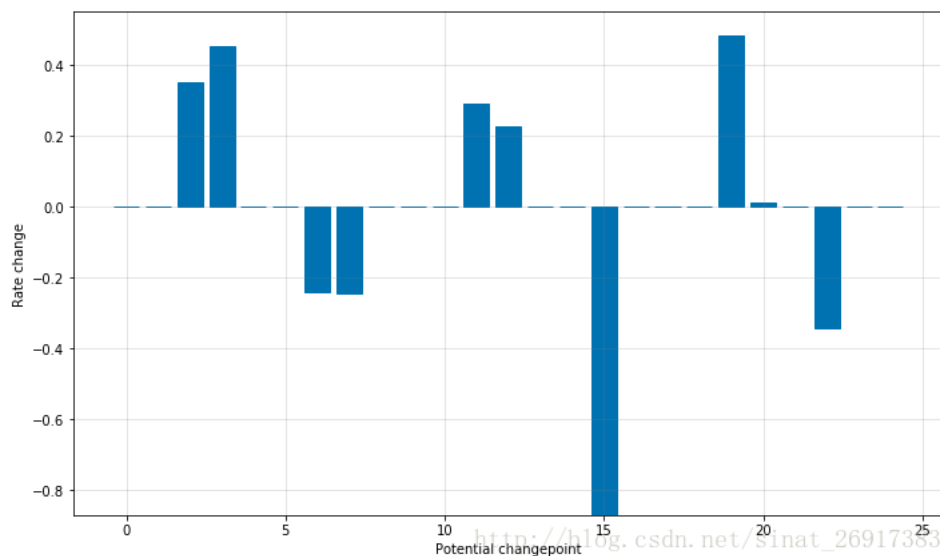
`changepoint.prior.scale`

会影响对趋势的判断，设置大些表示趋势受这些change point影响较大，过大当然可能导致过拟合。例如，我从0.01修改为0.1，就导致过拟合。如下图：





其自己检验突变点的方式，类似观察ARIMA的自相关/偏相关系数截尾、拖尾：

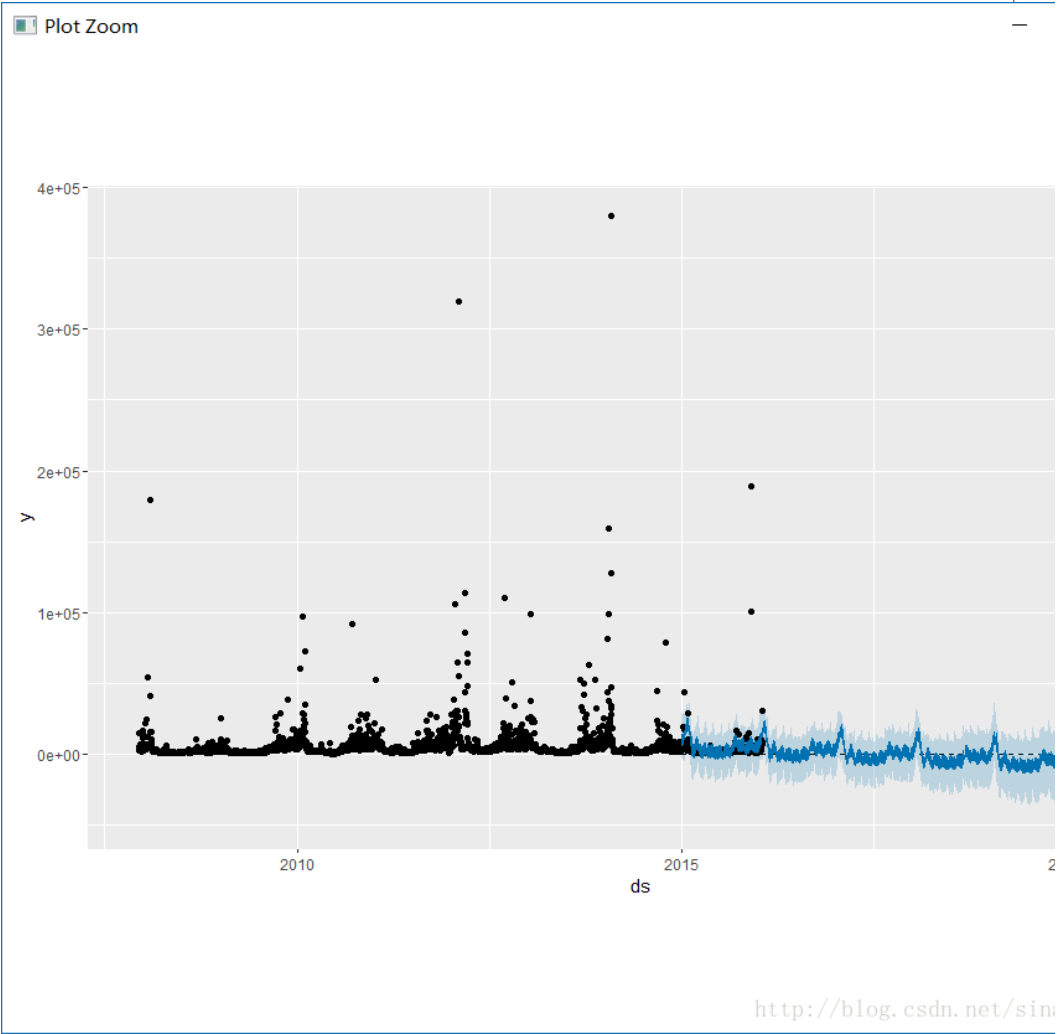


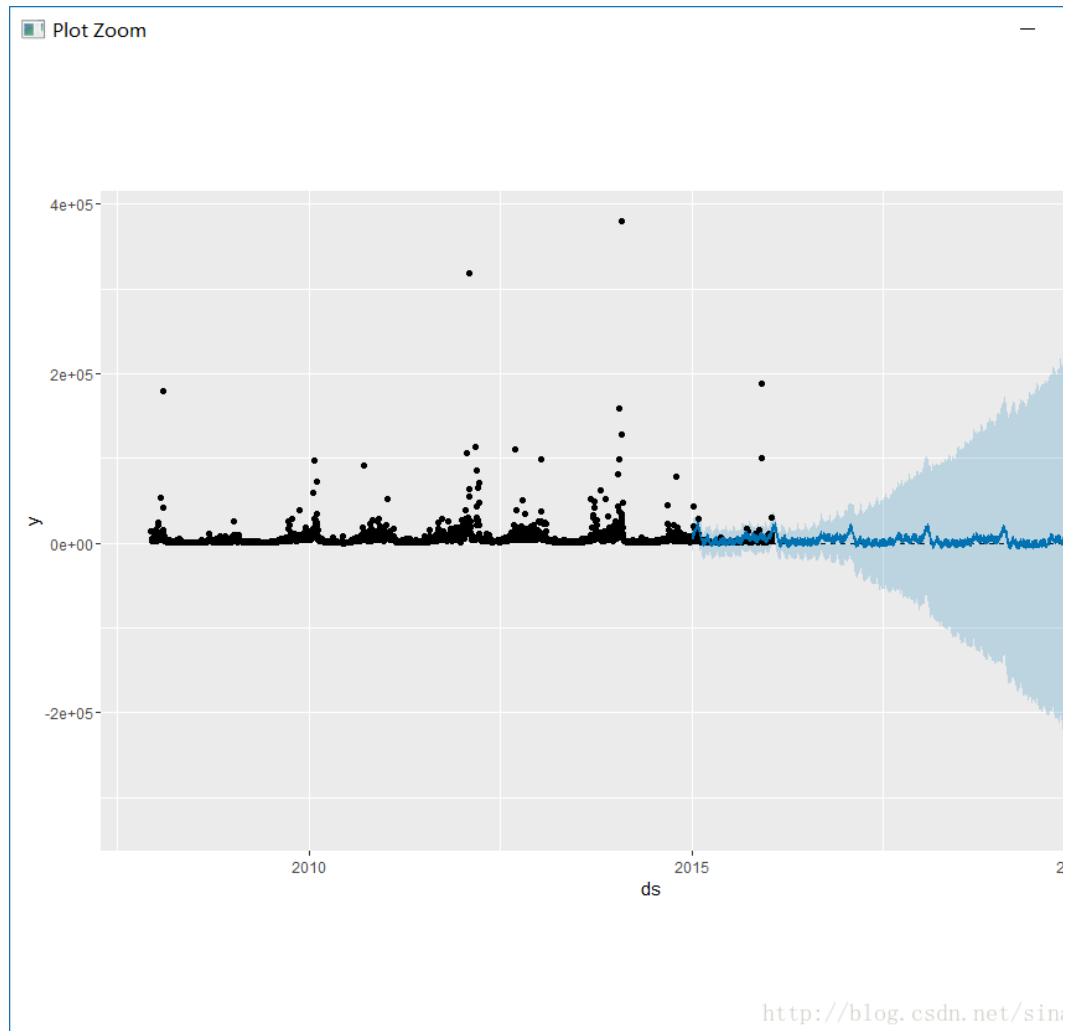
2、人为干预突变点——弹性范围

通过changeptprior_scale进行人为干预。

```
df = pd.read_csv('../examples/example_wp_peyton_manning.csv')
m <- prophet(df, changepoint.prior.scale = 0.5)
forecast <- predict(m, future)
plot(m, forecast)
```

来感受一下changeoint.prior.scale=0.05和0.5的区别：





<http://blog.csdn.net/sin>

可以把`changepoint.prior.scale`看成一个弹性尺度，值越大，受异常值影响越大，那么波动越大，如0.5这样的。

3、人为干预突变点——某突变点

当你知道数据中，存在某一个确定的突变点，且知道时间。可以用`changepoints` 函数。不po图了。

```
df = pd.read_csv('../examples/example_wp_peyton_manning.csv')
m <- prophet(df, changepoints = c(as.Date('2014-01-01')))
forecast <- predict(m, future)
plot(m, forecast)
```

4、突变预测

标题取了这么一个名字，也是够吓人的，哈哈~ 第三节的前3点都是如何消除突变点并进行预测。

但是！现实是，突变点是真实存在，且有些是有意义的，譬如双11、双12这样的节日。不能去掉这些突变点，但是不去掉又会影响真实预测，这时候Prophet新奇的来了一招：序列生成模型中，多少受异常值些影响（类似前面的`changepoint_prior_scale`，但是这里是从生成模型阶段就给一个弹性值）。

这里从生成模型中可以进行三个角度的调节：

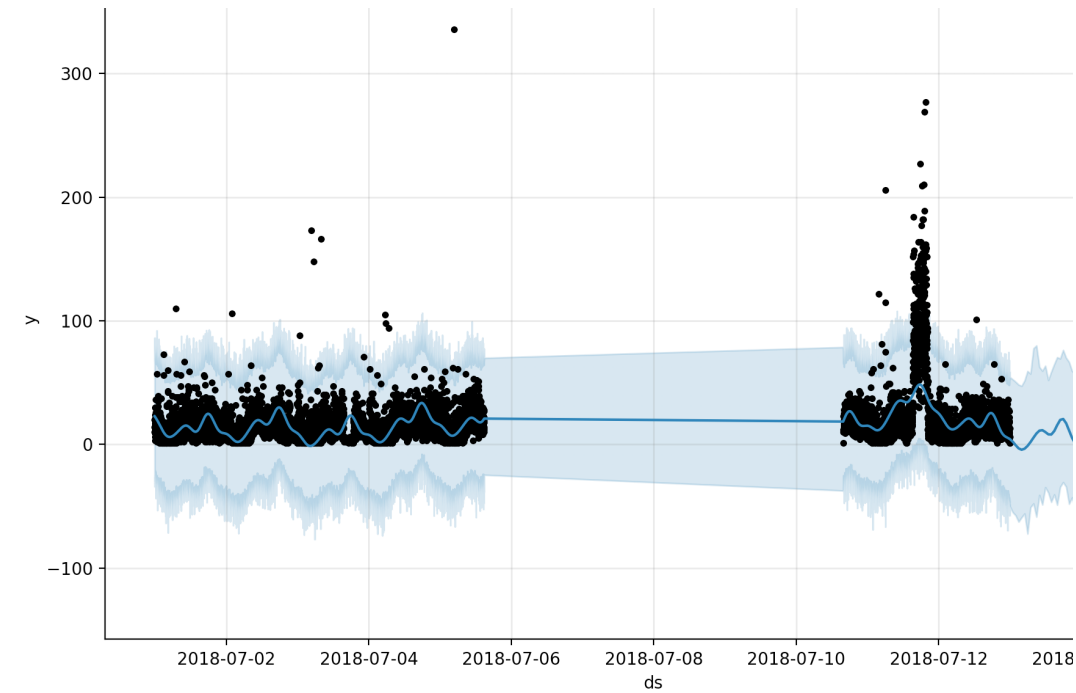
- (1) 调节趋势；
- (2) 季节性调节

- (1) 趋势突变适应

```
df = pd.read_csv('../examples/example_wp_peyton_manning.csv')
m <- prophet(df, interval.width = 0.95)
forecast <- predict(m, future)
```

在prophet生成模型阶段，加入`interval.width`，就是代表生成模型时，整个序列趋势，还有5%受异常值影响。

补充：在绘图看来就是上下界的曲线更宽了！！！！——本质上是更多考虑了趋势不确定性以及随机扰动的不确定性！！如果是季节不确定性的话，就要用到MCMC了！

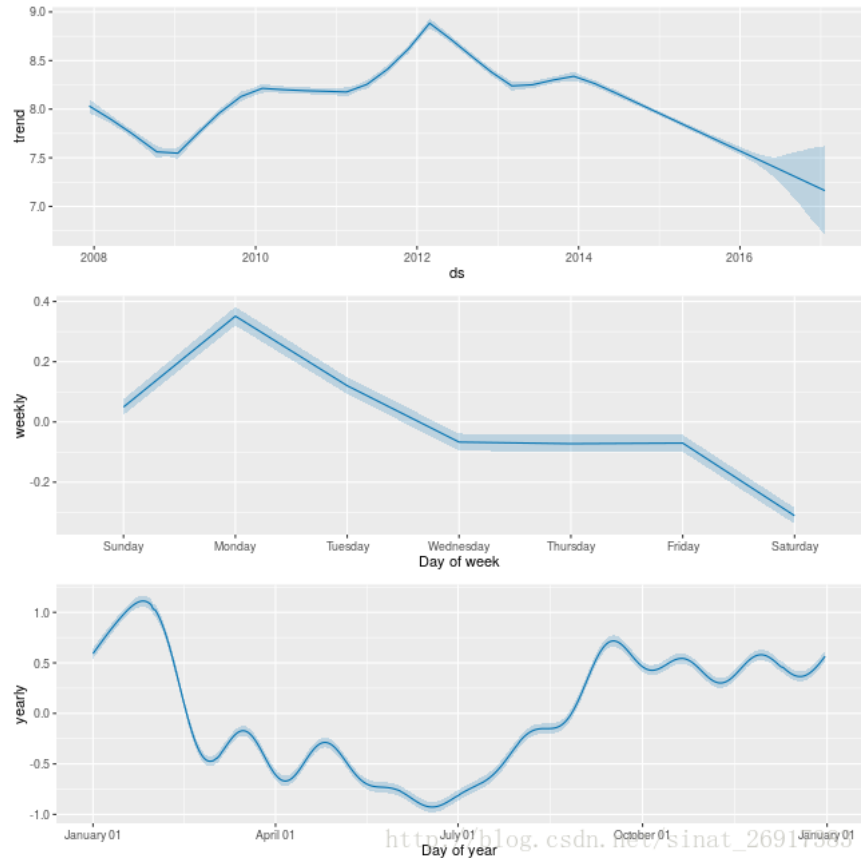


• （2）季节性突变适应

对于生产厂家来说，季节性波动是肯定有的，那么又想保留季节性突变情况，又要预测。而且季节性适应又是一个比较麻烦的事情，prophet里面需要先进行全贝叶斯抽样，mcmc.samples参数，默认为0。

```
m <- prophet(df, mcmc.samples = 500)
forecast <- predict(m, future)
prophet_plot_components(m, forecast);
```

打开mcmc.samples按钮，会把MAP估计改变为MCMC采样，训练时间很长，可能是之前的10倍。最终结果，官网DAO图：



补充：如上面所说，本质上就是引入了季节不确定性！

季节性的不确定性

默认情况下，先知只会返回不确定性的趋势和观测噪声。要获得季节性的不确定性，你必须做完全贝叶斯抽样。这是使用参数'MCMC_samples'（默认为0）来完成的。

这用MCMC采样代替了典型的MAP估计，并且可能需要更长的时间，这取决于有多少观测——期望几分钟而不是几秒钟。如果你做了完整的采样，那么当你绘制它们时，你会看到季节性成分的不确定性！

例子见：uncertainty_intervals.ipynb

关于MCMC，参考：<https://blog.csdn.net/google19890102/article/details/51755242>，我个人感觉本质上是在用HMM的思路来求解t+1时刻生成的样本问题。或许季节性的规律正是可以通过它来学习（需看fb prophet论文）

[MCMC](#)（Markov Chain Monte Carlo）是一种经典的概率分布采样方法。解决什么问题？

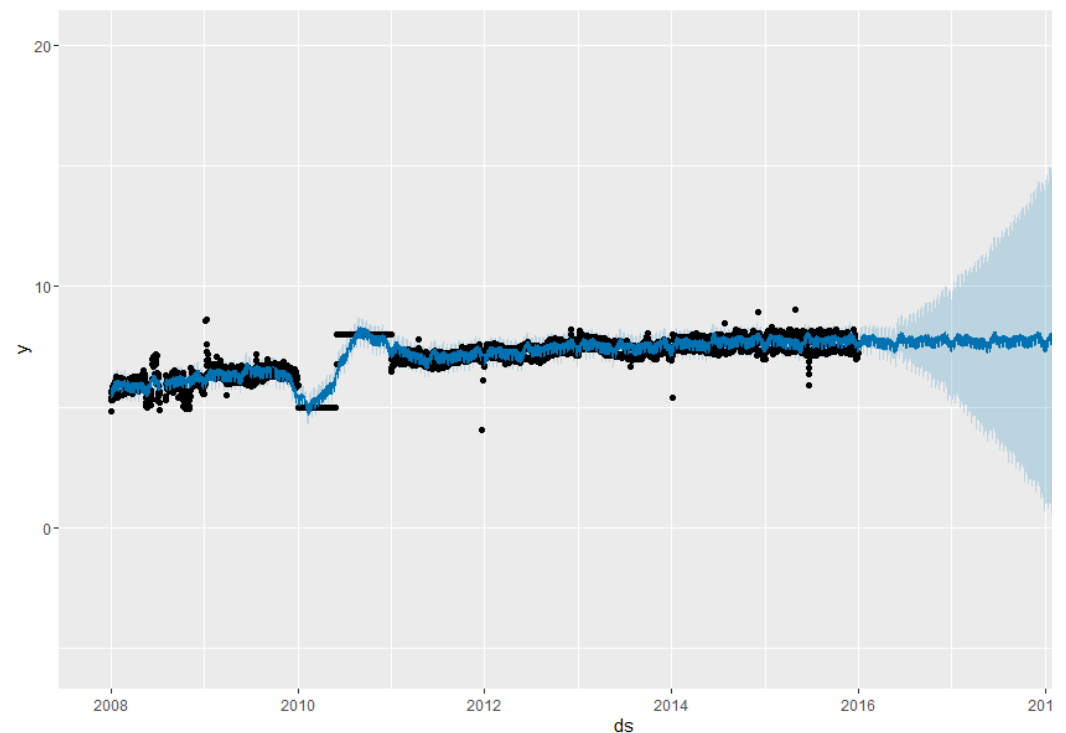
我们常常遇到这样的问题：模型构建好之后，有一个概率（称为目标分布），不能显式的给出其表达，只能生成一系列符合这个分布的。这种问题称为“采样”。

5、异常值/离群值

异常值与突变点是有区别的，离群值对预测影响尤其大。

```
df <- read.csv('../examples/example_wp_R_outliers1.csv')
df$y <- log(df$y)
m <- prophet(df)
future <- make_future_dataframe(m, periods = 1096)
forecast <- predict(m, future)
plot(m, forecast);
```

Plot Zoom

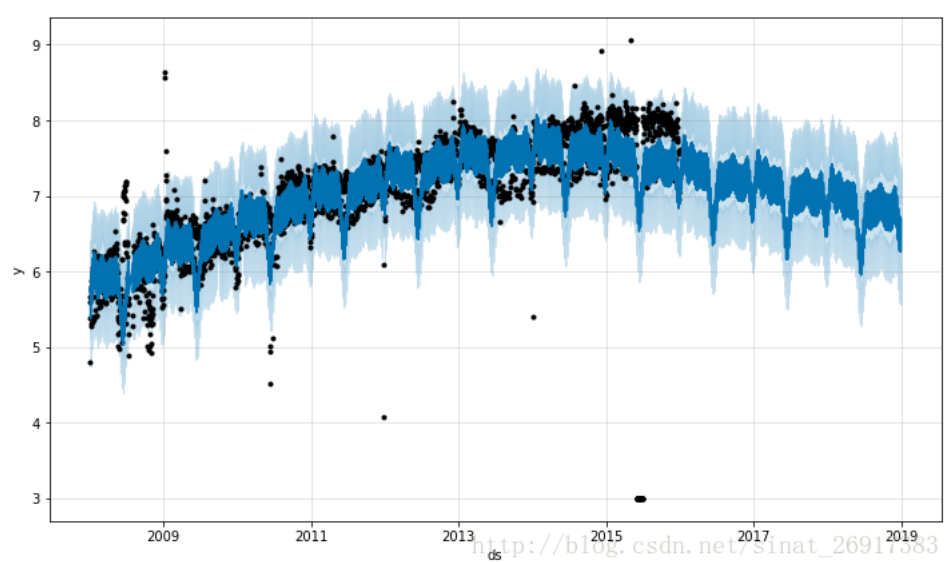


<http://blog.csdn.net/sin>

对结果的影响很大，而且导致预测置信区间扩大多倍不止。prophet的优势体现出来了，**prophet是可以接受空缺值NA的**，所以这些异常点删掉或者NA掉，都是可以的。

```
#异常点变为NA+进行预测
outliers <- (as.Date(df$ds) > as.Date('2010-01-01')
             & as.Date(df$ds) < as.Date('2011-01-01'))
df$y[outliers] = NA
m <- prophet(df)
forecast <- predict(m, future)
plot(m, forecast);
```

当然啦，你也可以删掉整一段影响数据，特别是天灾人祸的影响是永久存在的，那么可以删掉这一整段。下图就是这样的情况，2015年6月份左右的一批数据，都是离群值。



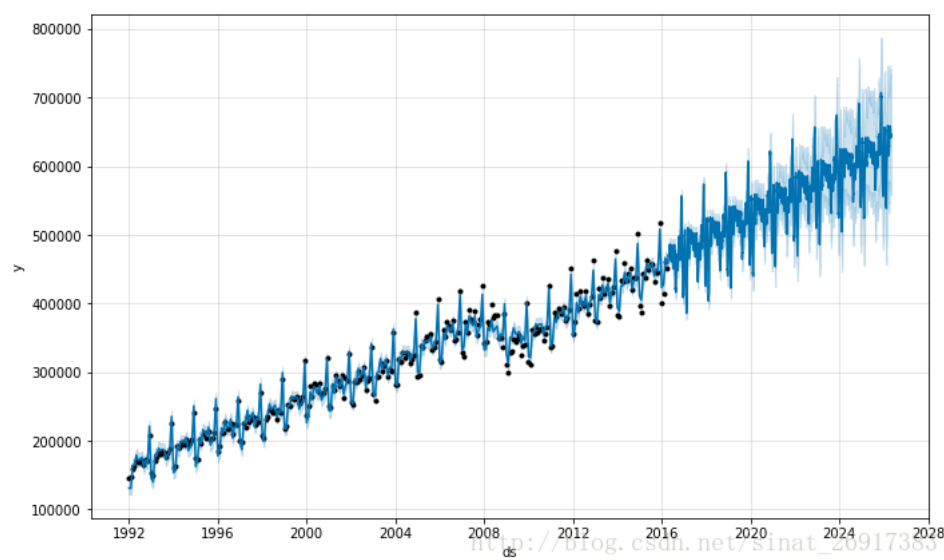
四、缺失值、空缺时间的处理+预测

前面第三章后面就提过，**prophet是可以处理缺失值**。那么这里就可以实现这么一个操作，如果你的数据不完整，且是间断的，譬如你有一个月20天的数据，那么你也可以根据prophet预测，同时给予你每天的数据结果。实现了以下的功能：

```
prophet=缺失值预测+插值

• 1

df <- read.csv('../examples/example_retail_sales.csv')
m <- prophet(df)
future <- make_future_dataframe(m, periods = 3652)
fcst <- predict(m, future)
plot(m, fcst);
```

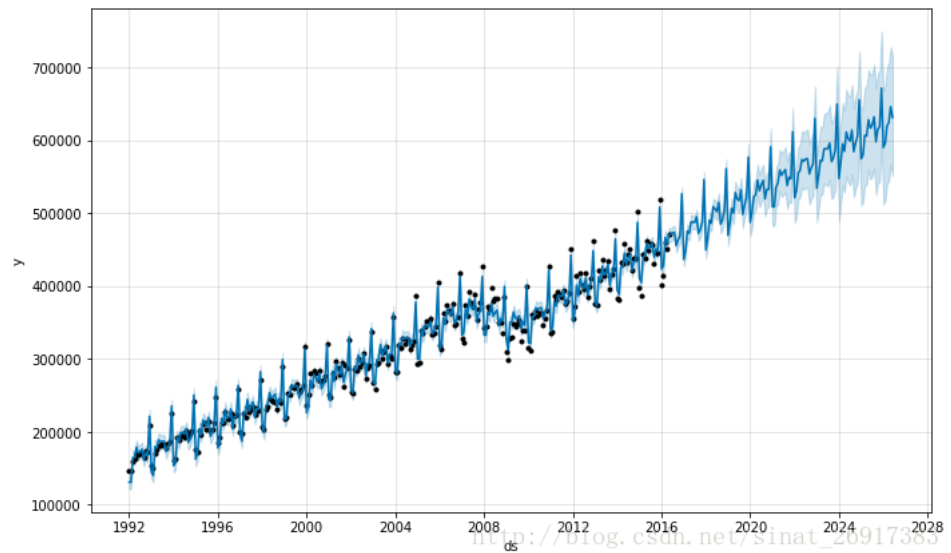


源数据长这样：

	ds	y
1	1992-01-01	146376
2	1992-02-01	147079
3	1992-03-01	159336
4	1992-04-01	163669
5	1992-05-01	170068

也就是你只有一年的每个月的数据，上面是预测接下来每一天的数据，也能预测，但是后面每天预测的误差有点大。所以你可以设置make_future_dataframe中的freq，后面预测的是每个月的：

```
future <- make_future_dataframe(m, periods = 120, freq = 'm')
fcst <- predict(m, future)
plot(m, fcst)
```



```
prophet = Prophet()
```

传入参数说明：（打开forecaster.py文件，查看它的__init__方法）

```
1. def __init__(
2.     self,
3.     growth='linear',
4.     changepoints=None,
5.     n_changepoints=25,
6.     yearly_seasonality=True,
7.     weekly_seasonality=True,
8.     daily_seasonality=True,
9.     holidays=None,
10.    seasonality_prior_scale=10.0,
11.    holidays_prior_scale=10.0,
12.    changepoint_prior_scale=0.05,
13.    mcmc_samples=0,
14.    interval_width=0.80,
15.    uncertainty_samples=1000,
16. );
```

growth = 'linear'：fbprophet有两种模式，线性增长模式和逻辑增长模式（growth='logistic'）

changepoints:指定潜在改变点，如果不指定，将会自动选择潜在改变点例如：

```
changepoints=['2014-01-01']
```

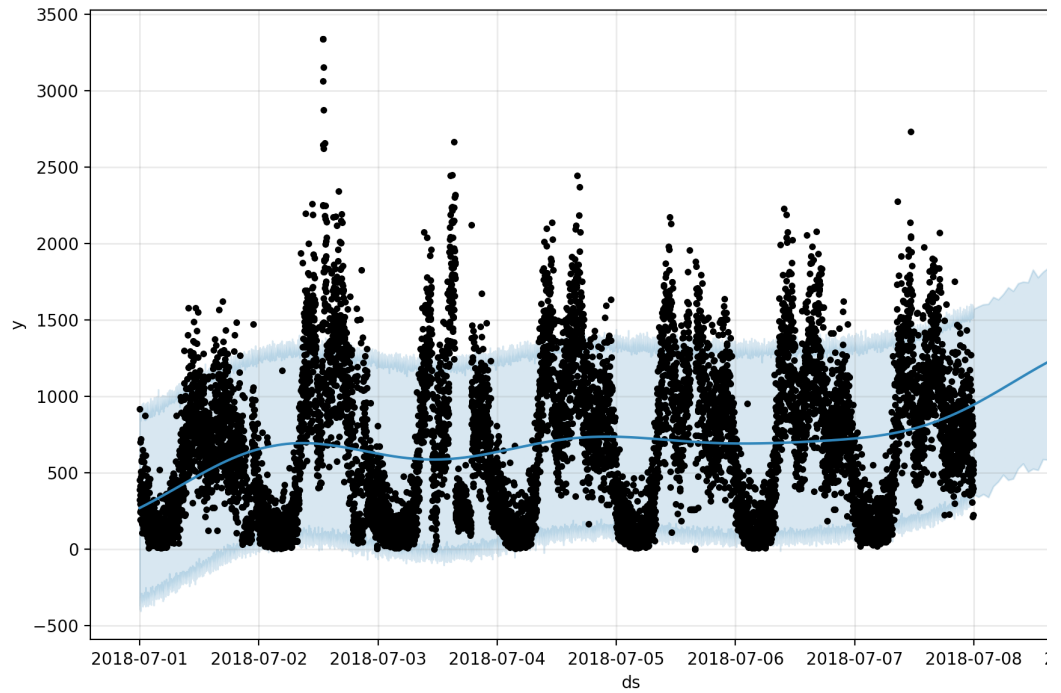
指定2014-01-01这一天是潜在的changepoints

n_changepoints:默认值为25，表示changepoints的数量大小，如果changepoints指定，该传入参数将不会被使用。如果changepoints不指定，将会从输入的历史数据前80%中选取25个（个数由n_changepoints传入参数决定）潜在改变点。

yearly_seasonality:指定是否分析数据的年季节性，如果为True,最后会输出，yearly_trend,yearly_upper,yearly_lower等数据。

weekly_seasonality:指定是否分析数据的周季节性，如果为True，最后会输出，weekly_trend,weekly_upper,weekly_lower等数据。

daily_seasonality:xxxxx,如下图，如果设置为false，则预测的时候表现不出数据每天相似的规律性。



holidays:传入pd.dataframe格式的数据。这个数据包含有holiday列 (string)和ds(date类型) 和可选列lower_window和upper_window来指定该日期的lower_window或者upper_window范围内都被列为假期。lower_window=-2将包括前2天的日期作为假期，例如：（这个是官方的quick start中的季后赛和超级杯两个假期的指定）

seasonality_prior_scale：季节性模型的调节强度，较大的值允许模型以适应更大的季节性波动，较小的值抑制季节性。

holidays_prior_scale:假期组件模型的调节强度。

changepoints_prior_scale:自动的潜在改变点的灵活性调节参数，较大值将允许更多的潜在改变点，较小值将允许更少的潜在改变点。

mcmc_samples:整数，若大于0，将做mcmc样本的全贝叶斯推理，如果为0，将做最大后验估计。指定贝叶斯抽样，例如mcmc_samples=20，

```
Iteration: 1 / 20 [ 5%] (Warmup) (Chain 0)
Iteration: 2 / 20 [ 10%] (Warmup) (Chain 0)
Iteration: 4 / 20 [ 20%] (Warmup) (Chain 0)
Iteration: 6 / 20 [ 30%] (Warmup) (Chain 0)
Iteration: 8 / 20 [ 40%] (Warmup) (Chain 0)
Iteration: 10 / 20 [ 50%] (Warmup) (Chain 0)
Iteration: 11 / 20 [ 55%] (Sampling) (Chain 0)
Iteration: 12 / 20 [ 60%] (Sampling) (Chain 0)
Iteration: 14 / 20 [ 70%] (Sampling) (Chain 0)
Iteration: 16 / 20 [ 80%] (Sampling) (Chain 0)
Iteration: 18 / 20 [ 90%] (Sampling) (Chain 0)
Iteration: 20 / 20 [100%] (Sampling) (Chain 0)
```

预测时会打印出这些。

interval_width:浮点数，给预测提供不确定性区间宽度，如果mcmc_samples=0,这将是预测的唯一不确定性，如果mcmc_samples>0，这将会被集成在所有的模型参数中，其中包括季节性不确定性

uncertainty_samples:模拟绘制数，用于估计不确定的时间间隔

make_future_dataframe() 准备预测的数据框架，该函数的定义如下：

```
1. def make_future_dataframe(self, periods, freq='D', include_history=True):
2. last_date = self.history['ds'].max()
3. dates = pd.date_range(
4. start=last_date,
5. periods=periods + 1, # closed='right' removes a period
6. freq=freq,
7. closed='right') # omits the start date
8.
9. if include_history:
10. dates = np.concatenate((np.array(self.history['ds']), dates))
11.
12. return pd.DataFrame({'ds': dates})
```

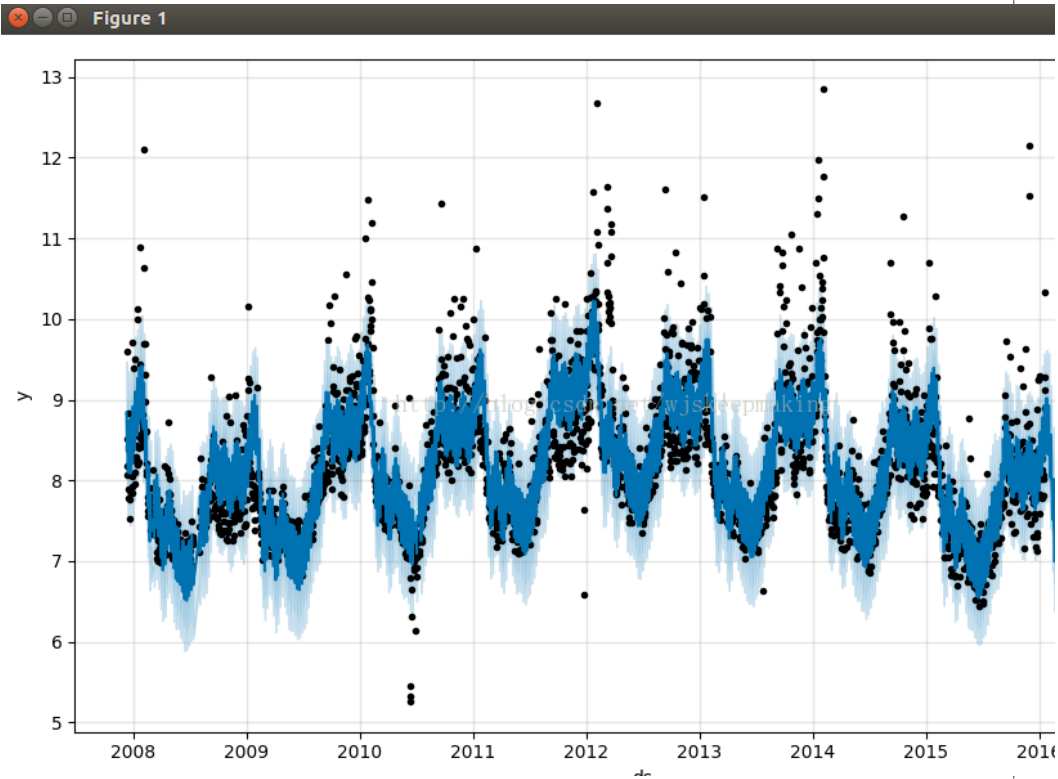
periods:指定要预测的时长，要根据freq这个参数去设置。假如freq='D'，那么粒度为天，periods则指定要预测未来多少天的数据。如果freq='H',则periods指定要预测未来多少小时的数据（当然，要在框架支持粒度为小时的前提下才能运用）。

include_history：是否包含历史数据，保持默认就好。

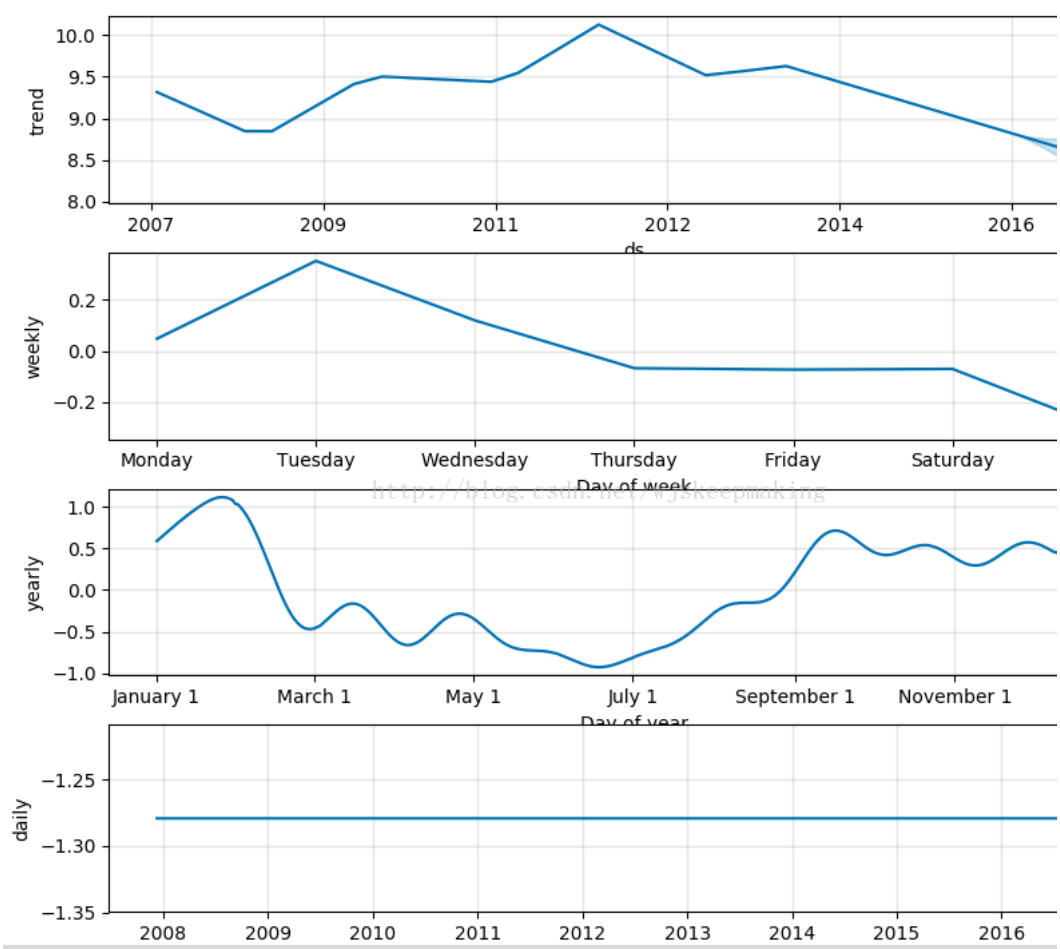
画图：

1. prophet.plot(forecasts).show()

2. prophet.plot_components(forecasts).show()

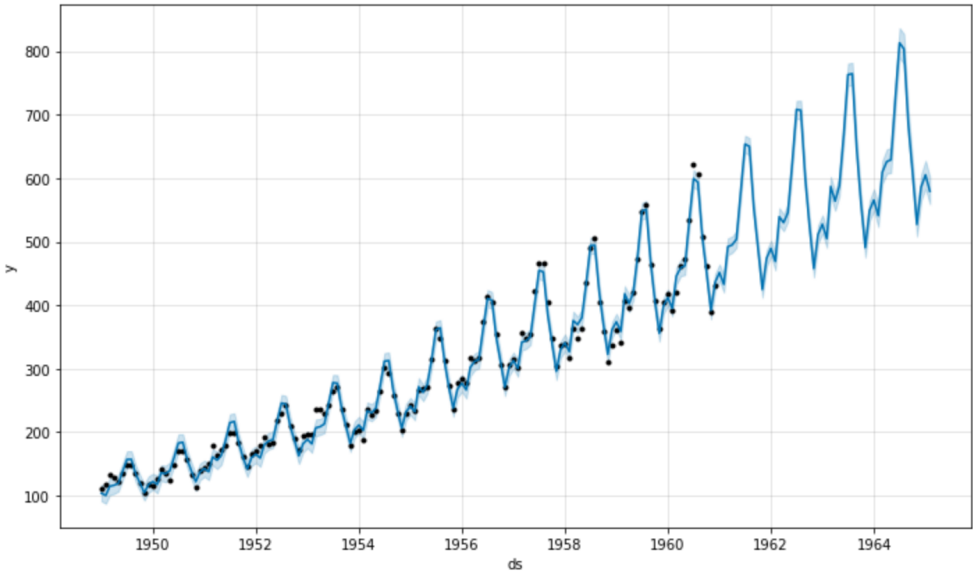


说明：图中浅蓝色表示yhat_upper和yhat_lower，受到interval_width这个参数的影响，当然，从forecasts的keys中可以看到，还有其他的_upper和_lower,同样也会受到这个参数的影响。



下面这个时间序列中，季节性不是先知所假设的常数加性因子，而是随趋势而增长。这是乘法季节性。此时要使用：
seasonality_mode='multiplicative'

```
i]: m = Prophet(seasonality_mode='multiplicative')
m.fit(df)
forecast = m.predict(future)
fig = m.plot(forecast)
```



对于季节性趋势变化不大的情形，使用的话容易过拟合：

